

Night-Hawk Docker 速查与排错记录

从 Dockerfile、镜像导出，到容器运行、端口映射和常见问题的实战整理

生成日期：2026-06-16 适用项目：Night-Hawk

一句话先记住：Dockerfile 是配方，docker build 才会生成镜像；镜像再被 docker compose up 或 docker run 运行成容器；而容器本质上就是一个隔离的普通进程。

- Dockerfile 负责描述 “怎么做” ；镜像是 “做出来的成品” ；容器是 “成品跑起来的实例” 。
- docker build 会读取 Dockerfile 并生成本地镜像；docker save / docker load 则负责镜像导出和导入。
- 当前项目里，服务对外访问端口要看宿主机映射；容器内部端口要和 Go 服务实际监听端口一致。

1. 命令速查

命令	用途
docker build -t go-order-service-app .	根据当前目录里的 Dockerfile 构建镜像；-t 是 tag，顺便给镜像命名。
docker images	查看本地已经存在的镜像列表。
docker save -o go-order-service-app.tar go-order-service-app:latest	把镜像导出成一个本地 tar 文件，便于离线搬运。
docker load -i go-order-service-app.tar	把 tar 文件重新导入为本地镜像。
docker compose up -d --build	按 docker-compose.yml 启动 mysql / redis / app，并在必要时重新构建 app 镜像。
docker compose ps	查看 Compose 管理的容器状态。
docker compose logs app	查看 app 容器日志，排查数据库、Redis 或启动失败问题。
docker ps	查看当前正在运行的容器。
docker compose config	展开并校验 Compose 配置，确认语法和最终生效内容。
curl.exe http://localhost:8500/api/v1/health	本地 go run 模式的健康检查地址。
curl.exe http://localhost:8500/api/v1/health/db	数据库健康检查。

命令	用途
curl.exe http://localhost:8500/api/v1/health/redis	Redis 健康检查。
curl.exe http://localhost:8500/api/v1/products	商品列表接口验证。

端口提醒：这台机器上最初尝试过 `8080:8500`，但 Windows 的排除端口范围把 8080 占掉了；最终可用的是 `8500:8500`，所以宿主机访问也用 `http://localhost:8500`。

2. 运行路径

- 1 进入项目根目录：cd D:\Workspace\Night-Hawk
- 2 构建并启动完整环境：docker compose up -d --build
- 3 确认容器都起来：docker ps 或 docker compose ps
- 4 查看 app 日志：docker compose logs app
- 5 访问健康检查：curl.exe http://localhost:8500/api/v1/health
- 6 跑接口回归：go run ./cmd/apitest health / db / redis / products / orders / payments / register / login / me

如果你想强制指定 apitest 地址，可以用 `-base`，例如：`go run ./cmd/apitest -base http://localhost:8500 health`。

3. 导出 / 导入 / 运行的关系

- Dockerfile -> docker build -> 镜像 (image)
- 镜像 (image) -> docker save -> tar 文件
- tar 文件 -> docker load -> 镜像 (image)
- 镜像 (image) -> docker compose up -> 容器 (container)
- 容器 (container) -> 宿主机端口映射 -> 浏览器 / curl 访问

4. 我们遇到的问题和原因

现象	原因	处理方式
docker compose up -d --build 报 no configuration file provided: not found	当前终端不在项目根目录，Compose 找不到 docker-compose.yml。	先 cd D:\Workspace\Night-Hawk，再执行 Compose 命令。

现象	原因	处理方式
docker build -t ... 报 open Dockerfile: no such file or directory	当前目录不是仓库根目录, Docker 找不到 Dockerfile。	切回项目根目录后再 build。
启动 app 时提示 ports are not available ... 8080	Windows 把 8080 放进了排除端口范围, 宿主机端口无法绑定。	把宿主机端口改成 8500:8500, 或换一个未被排除的端口。
failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine	Docker Desktop / Docker Engine 没有启动, CLI 连不到 daemon。	先启动 Docker Desktop, 再重试。
docker save -o go-order-service-app.tar ... 后没看到 .tar	-o 使用的是相对路径, 文件生成在你当前所在目录。	用 Get-Location 查当前目录, 或直接写绝对路径。
构建日志里出现 docker.io/library/...	docker.io 是镜像仓库地址 (registry), 不是 .iso 文件后缀。	镜像名看 docker images, 不要把它当成本地文件。
打开 Docker Desktop 后 app 没自动运行	Docker Desktop 只是引擎和管理界面, 容器需要 docker compose up 才会创建并启动。	手动执行 docker compose up -d --build。
访问 localhost:8500 连不上	要么本机没有 go run, 要么 Compose 宿主机端口不是 8500。	按当前实际端口访问; 这台机器最终用的是 8500:8500。

5. 快速结论

- 镜像不是 .iso; 它是 Docker 管理的对象。如果你需要文件, 可以用 docker save 导出成 .tar。
- 容器不是虚拟机, 它更像一个隔离的普通进程, 只是这个进程有自己的文件系统、网络和环境变量。
- 当前项目的正确思路是: 先 build 出镜像, 再用 Compose 把 mysql / redis / app 一起启动。以后再遇到“为什么命令不通 / 为什么端口不通 / 为什么看不到文件”这些问题, 先回到三个检查点: 当前目录、宿主机端口、Docker Engine 是否已启动。